



Tema 8

🕒 Tipo	Nota Apuntes
📎 Archivos y multimedia	DDD-v10E.pdf
📖 Bloques de notas	📖 Bloque 2

1. Introducción a DDD

Es un enfoque de desarrollo centrado en modelar el software de modo que su diseño coincida lo máximo posible con el dominio del negocio. Fue introducido por Eric Evans (Libro Azul, 2003), y más tarde por Vaughn Vernon (Libro Rojo, 2013). Un buen resumen es el del Libro Verde (2016), escrito por Vernon también.

Su objetivo es combatir la complejidad dividiendo problemas grandes en modelos manejables mediante la colaboración entre expertos de negocio y desarrolladores.

Se divide en Diseño Estratégico y Diseño Táctico.

2. Diseño Estratégico (El "Qué" y el "Dónde")

Se enfoca en entender el negocio y delimitar el problema antes de codificar. Equivalente a la estrategia militar, es decir, a gran escala.

- Core Domain (Dominio Principal): Es el diferenciador del negocio, lo que genera dinero y ventaja competitiva. Debe recibir el mayor esfuerzo de diseño.
- Generic Subdomains (Subdominios Genéricos): Necesarios para funcionar (ej. facturación) pero no son ventajas competitivas. Regla de oro: Evitar sobreinvertir en ellos.

- Lenguaje Ubicuo (Ubiquitous Language): Lenguaje estructurado y compartido por todo el equipo (expertos y técnicos) dentro de un contexto para eliminar ambigüedades.
- Bounded Context (Contexto Delimitado): Límite lógico (típicamente un subsistema o el trabajo de un equipo) dentro del cual un modelo concreto es aplicable y válido. Un término (como "Cliente") puede significar cosas distintas en diferentes contextos.

2.1 Mapa de Contextos (Context Map)

Define los patrones de relación entre múltiples Bounded Contexts.

- Open Host Service (OHS): El proveedor define un protocolo o API genérica para dar soporte a múltiples clientes.
- Published Language (PL): Lenguaje bien documentado (ej. esquema JSON/XML) que formaliza el OHS.
- Shared Kernel (Núcleo Compartido): Dos equipos comparten un subconjunto del modelo. Requiere mucha coordinación ante cambios.
- Conformist (Conformista): El equipo consumidor se adapta ciegamente al modelo del proveedor para minimizar el esfuerzo de traducción.
- Anti-Corruption Layer (ACL): Capa intermedia que traduce y aísla un modelo limpio de las "contaminaciones" de un sistema externo o legado.
- Customer/Supplier (Cliente/Proveedor): El proveedor tiene en cuenta las necesidades del cliente al diseñar. Hay reuniones y pruebas conjuntas.
- Separate Ways (Caminos Separados): Decisión deliberada de no integrarse por ser más barato desarrollar por cuenta propia.
- Big Ball of Mud (Gran Bola de Barro): Sistema legado monolítico sin límites definidos que debe ser segregado y aislado para que su "desastre" no se extienda.

3. Diseño Táctico: Bloques de Construcción

Una vez delimitado el contexto, se definen los elementos del modelo (Model-Driven Design).

- Entidades (Entities): Objetos definidos por su identidad y continuidad (ciclo de vida), no por sus atributos. Tienen un ID único aunque sus datos cambien (ej. una Cuenta Bancaria).
- Objetos de Valor (Value Objects - VO): Describen características sin identidad propia. Son inmutables; si cambian, se reemplaza el objeto entero. Dos VO son iguales si sus atributos son iguales (ej. un Color o una cantidad de Dinero).
- Agregados (Aggregates): Grupos de objetos (Entidades y VO) que se tratan como una unidad transaccional y comparten invariantes. Estos tienen una raíz del agregado (Aggregate Root), una única entidad que sirve como punto de entrada y hace cumplir las reglas de negocio.
- Repositorios (Repositories): Abstraen la persistencia. Simulan ser una colección de objetos en memoria para guardar y recuperar agregados sin que el dominio mencione bases de datos.
- Factorías (Factories): Encapsulan la lógica de creación compleja de Agregados o Entidades para no ensuciar el código del objeto.
- Eventos de Dominio (Domain Events): Anuncian un cambio de estado significativo que ya ocurrió (en pasado, ej. "PedidoPagado") para que otros componentes reaccionen.
- Servicios de Dominio (Domain Services): Lógica pura de negocio que no encaja en una sola entidad y suele cruzar varias. No confundir con servicios de aplicación.

4. Arquitectura y Conclusiones

- Arquitectura en Capas: Presentación, Aplicación, Dominio (donde vive el MDD) e Infraestructura.
- Aislamiento: DDD exige aislar la infraestructura (redes, BD) del "cerebro" o núcleo del software (el modelo del dominio).
- Perspectiva estratégica: Primero se centra en el Core Domain, se usa el lenguaje ubicuo y se protegen las fronteras (Bounded Contexts) antes de pasar al enfoque táctico de código.